

Mock 2 — In-Memory Database

ICF house style, 4 levels, 200-600

In-Memory Database — Coding Assessment

Your task is to implement a simplified in-memory database. All operations that should be supported are listed below. Partial credit will be granted for each test passed, so run `python tests.py` often to receive partial credit for passed tests. Please check tests for requirements and argument types.

Implementation Tips Read the question all the way through before you start coding, but implement the operations and complete the levels one by one, not all together, keeping in mind that you will need to refactor to support additional functionality. Please, do not change the existing method signatures.

Time limit: **90 minutes**. Score: 200-600.

Level 1 — Initial Design & Basic Functions

Records are stored under a **key**, and each record holds any number of **fields** with integer values.

- `set(key, field, value)`
 - Insert or update the field's value for the given key.
 - Returns nothing.
- `get(key, field)`
 - Return the value of the field, or `None` if the key or field doesn't exist.
- `delete(key, field)`
 - Remove the field from the record. Returns `True` if the field existed and was removed, `False` otherwise.
- `compare_and_set(key, field, expected, new_value)`
 - Set the field to `new_value` **only if** its current value equals `expected`. Returns `True` if the update happened, `False` otherwise.
 - If the field does not exist, returns `False`.

Level 2 — Data Structures & Data Processing

- `scan(key)`
 - Return a list of strings "`<field>(<value>)`" for every field of the record, sorted alphabetically by field name.
 - Returns an empty list if the key doesn't exist.
- `scan_by_prefix(key, prefix)`
 - Same as `scan`, but only fields whose name starts with `prefix`.
- `top_n_keys(n)`
 - Return a list of strings "`<key>(<field_count>)`" for the `n` keys with the most fields, ordered by field count descending, ties broken by key name ascending (lexicographically).

- If fewer than n keys exist, return all of them.

Level 3 — Refactoring & Encapsulation

Records may now have a limited lifespan. Implement timestamped variants of the existing operations. These **inherit all functionality** of the originals and additionally take a timestamp, and setters may specify a `ttl` — no `ttl` means the field lives forever.

A field set at time t with `ttl x` is available for queries with timestamp in $[t, t + x)$. At $t + x$ it has expired.

- `set_at(timestamp, key, field, value)`
- `set_at(timestamp, key, field, value, ttl)` — same method, optional argument
- `get_at(timestamp, key, field)`
- `delete_at(timestamp, key, field)`
- `scan_at(timestamp, key)`
- `scan_by_prefix_at(timestamp, key, prefix)`

Only fields that are still alive at timestamp are visible to any `_at` query. Re-setting an existing field replaces its value **and** resets its lifespan from the new timestamp.

The Level 1 and Level 2 methods must keep working exactly as before.

Level 4 — Extending Design & Functionality

- `rollback(timestamp)`
 - Restore the database to the state it had at the given timestamp.
 - All `ttls` must be recalculated relative to the operations that created them — a field set at time 5 with `ttl 10` is still alive after a rollback to time 8, and expires at 15 as it originally would.
 - Operations that happened after timestamp are discarded permanently.

Notes

- Standard library only.
- Determinism matters: every ordering rule above is exact.
- The grader is `tests.py`; implement in `database.py`.